# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO2 | **Assessment:** | Assessment Tool 2 – Architecture Design Assignment |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO2 Statement:** | *Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills* | | |
| **Student Name:** | S. Jefna Princy | **Reg. No.:** | 192572058 |
| **Date:** | 5/08/2026 | **Duration:** | 60 minutes |

**Code of Conduct**

*I* S. Jefna Princy 192572058 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

## Annexure A – Architecture Design Assignment

**Instructions to Students:**

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

## Rubrics for Calculation

| Evaluation Criteria | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **System Requirement Analysis** | Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture. | Analyzes most requirements with minor omissions or limited explanation. | Identifies only basic requirements with partial analysis. | Requirement analysis is incomplete, inaccurate, or lacks clarity. |
| **Selection of Software Architecture** | Selects the most appropriate architectural style with strong technical justification aligned to system requirements. | Selects a suitable architecture with reasonable justification. | Architecture is partially suitable with limited justification. | Inappropriate architecture selected or justification is missing. |
| **Architecture Diagram and Component Design** | Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented. | Diagram is mostly complete with minor omissions or notation issues. | Diagram includes only essential components and lacks sufficient detail. | Diagram is incomplete, unclear, or technically incorrect. |

| | | | | |
|---|---|---|---|---|
| **Component Interaction and Quality Attribute Analysis** | Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability. | Explains most components and quality attributes with minor gaps. | Provides limited explanation of interactions and addresses only some quality attributes. | Component interactions and quality attributes are poorly explained or missing. |
| **Architectural Justification and Technical Presentation** | Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report. | Provides adequate justification with minor gaps; report is well organized. | Limited justification with minimal discussion of trade-offs or future scope; report needs improvement. | Justification is weak or absent; report lacks organization and technical clarity. |

| Criteria | **Mark** |
|---|---|
| System Requirement Analysis | /5 |
| Selection of Software Architecture | /5 |
| Architecture Diagram and Component Design | /5 |
| Component Interaction and Quality Attribute Analysis | /5 |
| Architectural Justification and Technical Presentation | /5 |
| **TOTAL** | /5 |

**Course code: CSA1013** **Name: S Jefna Princy**

**Course: Software Engineering** **Reg No: 192572058**

# CO 2 ASSESSMENT TOOL - 2

# ARCHITECTURE DESIGN ANALYSIS

## 1. System Requirement Analysis

### 1.1 Industry Context

Rapid urbanization has significantly increased noise pollution due to road traffic, construction activities, industries, railway stations, and public events. Continuous exposure to excessive noise affects public health by causing stress, sleep disorders, hearing impairment, and reduced productivity. Traditional manual monitoring methods cannot provide continuous real-time information. Therefore, an AI-driven smart noise pollution monitoring system is required to monitor noise levels continuously, identify high-risk zones, and assist authorities in taking immediate corrective actions.

### 1.2 Problem Statement

Existing noise monitoring systems rely on manual measurements taken periodically, making it difficult to detect sudden changes in urban noise levels. This results in delayed identification of noise hotspots and ineffective enforcement of environmental regulations. The proposed AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System uses IoT noise sensors, cloud computing, artificial intelligence, and GIS mapping to monitor sound levels in real time, predict future noise trends, identify affected areas, and automatically alert authorities whenever permissible noise limits are exceeded.

### 1.3 Functional Requirements

- Collect real-time noise data using IoT sensors.
- Store sensor data securely in cloud databases.
- Analyze noise levels using Artificial Intelligence.
- Detect abnormal noise patterns.
- Display noise hotspots on GIS maps.
- Generate automatic alerts when noise exceeds limits.
- Produce environmental monitoring reports.
- Allow authorities to monitor and manage the system through a dashboard

### 1.4 Non-Functional Requirements

- High system availability (24×7 operation).
- Fast response for real-time monitoring.
- Secure storage and transmission of data.
- Scalable cloud infrastructure to support thousands of sensors.
- Reliable data collection with minimal downtime.
- Easy maintenance and future upgrades.
- User-friendly interface for administrators and environmental officers.

### 1.5 Quality Attributes

- **Scalability:** Supports increasing numbers of IoT sensors and users.
- **Performance:** Processes real-time data with low latency.
- **Security:** Protects environmental data through authentication and encryption.
- **Reliability:** Ensures continuous monitoring without interruption.
- **Availability:** Provides uninterrupted access to authorized users.
- **Maintainability:** Modular architecture allows easy updates and maintenance.

## 2. Selection of Suitable Software Architecture

### 2.1 Selected Software Architecture

The **Layered Architecture (Three-Tier Architecture)** is selected for the **AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System** because it provides a clear separation of responsibilities, improves maintainability, and supports scalability.

The system is divided into three main layers:

**1. Presentation Layer**

- User Dashboard
- Environmental Officer Dashboard
- Authority Dashboard
- GIS Map Interface
- Report Generation Interface

**2. Application Layer**

- User Authentication Module
- Noise Data Collection Module

- AI Noise Analysis Module
- GIS Mapping Module
- Alert & Notification Module
- Report Generation Module

**3. Data Layer**

- Cloud Database
- Sensor Data Storage
- Historical Noise Records
- AI Model Storage

**2.2 Why Layered Architecture is Suitable**

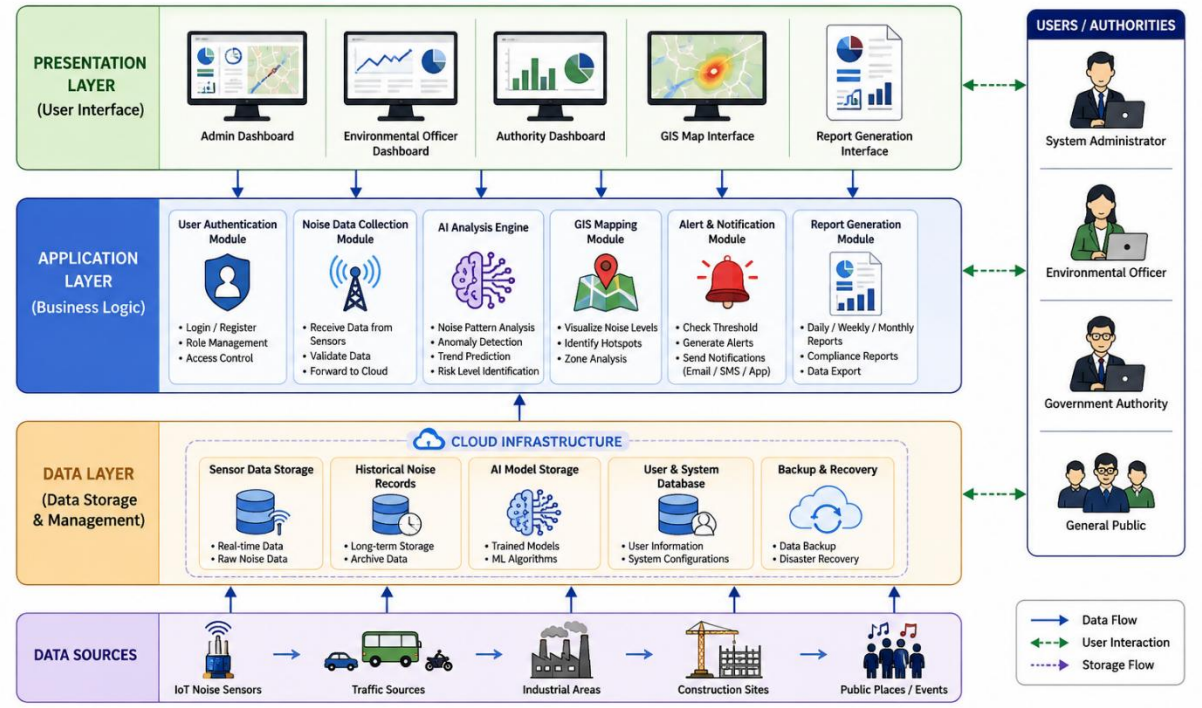
The Layered Architecture is the most appropriate choice because:

- It separates the user interface, business logic, and database into independent layers.
- It simplifies system maintenance and future upgrades.
- New features can be added without affecting the entire system.
- It supports integration with AI, IoT sensors, cloud services, and GIS technologies.
- It improves system security by controlling access between different layers.
- It enables efficient processing of large volumes of real-time sensor data.

**2.3 Advantages of the Selected Architecture**

- Easy to understand and implement.
- Highly scalable for smart city applications.
- Better security through layered access control.
- Easier debugging and testing.
- Supports cloud-based deployment.
- Improves system performance and reliability.
- Allows independent modification of each layer.
- Reduces maintenance cost and development time.

## 3. Software Architecture Diagram




### 3.1 Components of the Architecture

#### 1. IoT Noise Sensors

IoT-based noise sensors are installed at different urban locations to continuously measure environmental noise levels. These sensors collect real-time sound intensity data and send it to the cloud.

#### 2. Cloud Storage

The cloud stores all incoming sensor data securely. It maintains historical records, supports large-scale data storage, and enables easy access for analysis.

#### 3. AI Analysis Engine

The AI engine analyzes the collected noise data using machine learning algorithms. It identifies abnormal noise patterns, predicts future trends, and detects high-risk noise zones.

#### 4. GIS Mapping System

The GIS module displays noise pollution data on interactive digital maps. It helps authorities identify hotspots and monitor affected regions geographically.

#### 5. Alert & Notification System

When noise levels exceed the permissible limit, the system automatically generates alerts and sends notifications to environmental authorities for immediate action.

**6. User Dashboard**

The dashboard provides real-time monitoring, graphical reports, GIS maps, analytics, and system status. Authorized users can easily view and manage the entire system.

**7. Environmental Authorities**

Authorities access the dashboard to monitor urban noise pollution, receive alerts, analyze reports, and make informed decisions for pollution control.

**3.2 Working of the System**

1. IoT sensors continuously collect environmental noise data.
2. The collected data is transmitted to the cloud database.
3. The AI engine processes and analyzes the data.
4. GIS mapping displays the locations of noise hotspots.
5. If the noise exceeds the threshold, alerts are generated automatically.
6. Environmental authorities access the dashboard to monitor reports and take corrective actions.

**4. Explain Components and Their Interactions**

**4.1 Purpose and Responsibilities of Each Component**

**1. IoT Noise Sensors**

**Purpose:** Continuously monitor environmental noise levels in different urban areas.

**Responsibilities:**

- Measure real-time sound intensity.
- Collect noise data from roads, industries, schools, hospitals, and residential areas.
- Transmit data to the cloud server.

**2. Cloud Storage**

**Purpose:** Store and manage all collected noise data securely.

**Responsibilities:**

- Store real-time and historical noise data.
- Ensure secure and reliable data storage.
- Provide data for AI analysis and report generation.

**3. AI Analysis Engine**

**Purpose:** Analyze collected noise data intelligently.

**Responsibilities:**

- Detect abnormal noise patterns.
- Predict future noise pollution trends.
- Identify high-risk noise zones.
- Support decision-making using machine learning algorithms.

**4. GIS Mapping Module**

**Purpose:** Display the geographical distribution of noise pollution.

**Responsibilities:**

- Show noise hotspots on digital maps.
- Visualize pollution intensity in different locations.
- Help authorities identify affected areas easily.

**5. Alert and Notification Module**

**Purpose:** Notify authorities whenever excessive noise levels are detected.

**Responsibilities:**

- Compare noise levels with permissible limits.
- Generate automatic alerts.
- Send notifications through dashboards, email, or SMS.

**6. User Dashboard**

**Purpose:** Provide a centralized interface for monitoring and management.

**Responsibilities:**

- Display real-time sensor data.
- Show AI analysis results.
- Display GIS maps and reports.
- Monitor system status and alerts.

**7. Environmental Authorities**

**Purpose:** Monitor environmental conditions and take necessary action.

**Responsibilities:**

- Review alerts and reports.

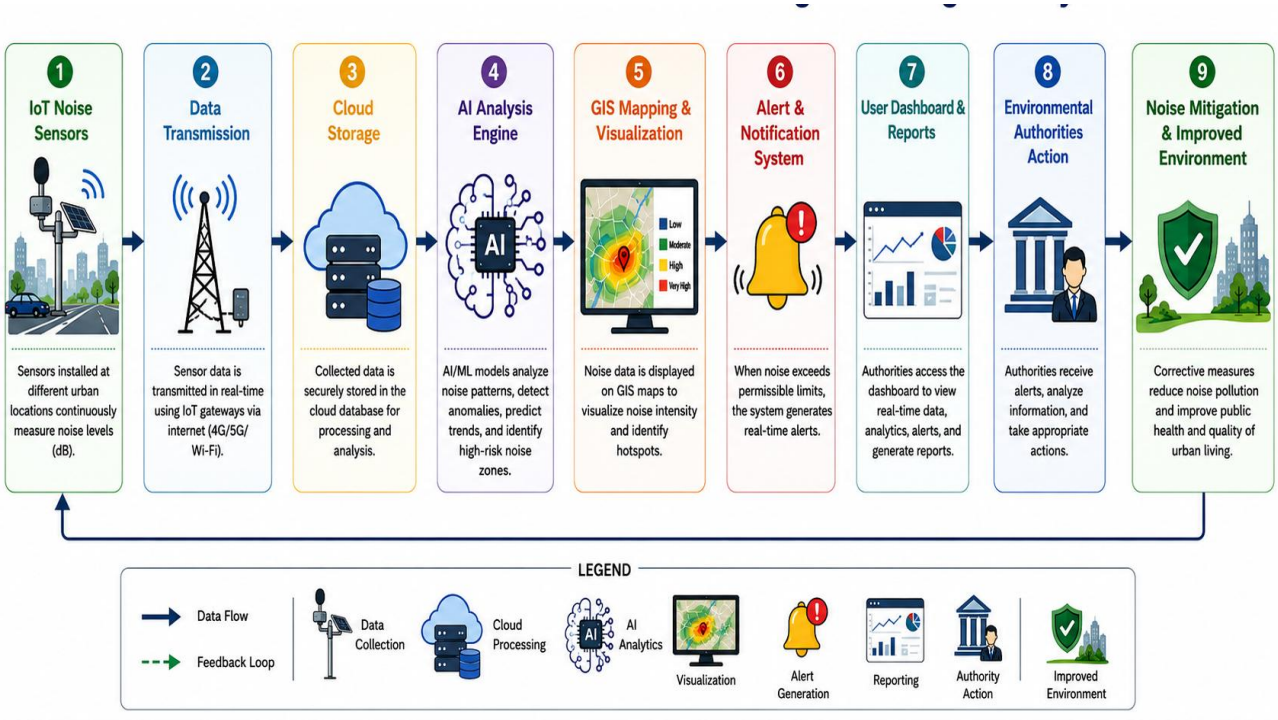
- Investigate high-noise areas.
- Enforce environmental regulations.
- Plan noise mitigation strategies.

### 4.2 Component Interaction

The interaction among system components is as follows:

1. IoT noise sensors continuously collect environmental sound data.
2. The collected data is transmitted securely to the cloud storage.
3. The AI Analysis Engine processes the stored data to identify abnormal noise patterns and predict future trends.
4. The processed information is displayed on the GIS Mapping Module, highlighting high-risk noise zones.
5. If the detected noise exceeds the predefined threshold, the Alert and Notification Module automatically sends notifications to environmental authorities.
6. Authorities access the User Dashboard to monitor real-time data, analyze reports, review GIS maps, and take corrective actions to reduce noise pollution.

4.3 Overall Workflow of the System



## 5. Discuss Quality Attributes

Quality attributes describe the characteristics that determine the overall performance, reliability, and effectiveness of the proposed system. The **AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System** is designed to satisfy the following quality attributes.

### 5.1 Scalability

The system is designed to support a growing number of IoT noise sensors deployed across different urban locations. As the city expands, new sensors and monitoring stations can be added without affecting the performance of the existing system. Cloud-based infrastructure enables the system to handle increasing volumes of real-time data efficiently.

### 5.2 Security

Security is an essential quality attribute because the system handles environmental monitoring data and administrative information. User authentication, role-based access control, secure communication protocols, and data encryption are implemented to prevent unauthorized access and ensure data confidentiality and integrity.

### 5.3 Performance

The system provides real-time monitoring and analysis of urban noise levels with minimal delay. AI algorithms process incoming sensor data efficiently, allowing rapid detection of abnormal noise levels and quick generation of alerts. Optimized cloud services ensure fast response times and smooth dashboard performance.

### 5.4 Reliability

The proposed system is highly reliable and capable of continuous operation throughout the day. Cloud storage, backup mechanisms, and fault-tolerant system components help ensure uninterrupted monitoring even in the event of hardware or network failures.

### 5.5 Maintainability

The system follows a modular architecture, allowing individual components such as the AI engine, GIS module, cloud database, or alert system to be updated independently. This simplifies maintenance, reduces downtime, and supports future enhancements without affecting the complete system.

### 5.6 Availability

The system is designed for **24×7 availability**, enabling continuous monitoring of environmental noise. Authorized users, including environmental officers and government authorities, can access the dashboard anytime to monitor real-time data, receive alerts, and generate reports whenever required.

## Summary of Quality Attributes

| Sl. No. | Quality Attribute | Description |
|---|---|---|
| 1. | Scalability | Supports additional IoT sensors and users without affecting system performance. |
| 2. | Security | Protects data through authentication, encryption, and secure communication. |
| 3. | Performance | Processes real-time noise data quickly and generates alerts with minimal delay. |
| 4. | Reliability | Ensures continuous monitoring with backup and fault tolerance. |
| 5. | Maintainability | Modular design allows easy updates and future improvements. |
| 6. | Availability | Provides uninterrupted 24×7 access to monitoring services and dashboards. |

### 6. Justify Architectural Decisions

The proposed **Layered Software Architecture** was selected because it provides a clear separation of responsibilities among different system components, making the application easier to design, develop, maintain, and upgrade. Each layer performs a specific function while communicating only with adjacent layers, which improves modularity and system organization.

The **Presentation Layer** provides a user-friendly interface through which environmental officers, municipal authorities, and administrators can monitor real-time noise levels, visualize GIS maps, receive alerts, and generate reports.

The **Application Layer** contains the business logic of the system. It manages sensor data processing, AI-based noise analysis, GIS mapping, alert generation, and report creation. This layer acts as the bridge between the user interface and the database.

The **Data Layer** securely stores sensor readings, AI prediction results, GIS location data, user information, and historical environmental records in a cloud database. It ensures reliable data storage and quick retrieval.

**Trade-offs Considered**

Although a layered architecture may introduce a slight communication delay between layers, the benefits significantly outweigh this limitation. The architecture provides:

- Better code organization and modularity.
- Easier testing and debugging.

- Simplified maintenance and future upgrades.
- Improved system reliability and scalability.
- Independent modification of each layer without affecting the entire application.

**Support for Future Enhancements**

The selected architecture allows future improvements without major redesign. New features such as:

- Advanced AI prediction models
- Mobile application support
- Integration with Smart City platforms
- Additional IoT sensor networks
- Real-time citizen reporting
- Government environmental databases